大模型常见面试题

基础理论和模型框架

1、Transformer核心机制

1. **概述一下transformer的架构**

答：transformer是基于自注意力机制的模型架构，核心是编码器-解码器结构，主要用于处理序列数据。

自注意力机制摆脱了RNN对序列的依赖，实现了并行计算，大幅度提升了长序列的处理能力。

a. 编码器（encoder）：由N个相同的编码层堆叠，每个编码层包含两个子层

   i. 多头自注意力：通过多个并行的注意力头，计算输入序列的的每个token与其它所有token之间的关联权重，捕获全局依赖关系。

   ii. 前馈神经网络：对每个token进行非线形变换。

b. 解码器（decoder）：由N个相同的解码层堆叠，每个解码层包含三个子层

   i. 掩码多头自注意力：掩码让生成序列时仅依赖已生成的token(避免未来信息泄漏)

   ii. 编码器-解码器注意力：关注编码器输出的关键信息。

   iii. 前馈神经网络：同上

   iv. 每个子层都有残差连接和层归一化（layer normalization）,增强训练稳定性。

2. **手撕代码（自注意力，多头自注意力）**

self-attention

```python
import torch
import torch.nn as nn
from math import sqrt
class SelfAttention(nn.Module):
    def __init__(self,input_dim,dim_q_k,dim_v):
        self.q = nn.Linear(input_dim,dim_q_k)
        self.k = nn.Linear(input_dim,dim_q_k)
        self.v = nn.Linear(input_dim,dim_v)
        self.norm = sqrt(dim_q_k)

    def forward(self,x,mask=None):
        q = self.q(x)
```

```
        k = self.k(x)
        v = self.v(x)

        #注意力分数计算
        d = torch.matmul(q,k.transpose(-2,-1))/self.norm

        #处理mask
        if mask is not None:
            d = d.masked_fill(mask==0,-1e9)

        #计算注意力分布
        softmax_out = torch.softmax(d,dim=-1)
        attention_out = torch.matmul(softmax_out,v)
        return attention_out
```

multiHeadAttention

```
class MultiHeadSelfAttention(nn.Module):
    def __init__(self,input_dim,head,dim_q_k,dim_v):
        self.head = head
        self.dim_per_head_q_k = dim_q_k       #每个头的query-key维度
        self.dim_per_head_v = dim_v            #每个头的value维度

        self.q = nn.Linear(input_dim,dim_q_k)
        self.k = nn.Linear(input_dim,dim_q_k)
        self.v = nn.Linear(input_dim,dim_v)

        self.norm = sqrt(self.dim_per_head_q_k)     #缩放因子
        self.out = nn.Linear(dim_v,input_dim)     # 输出全连接层将结果映射回 input_dim

    def forward(self,x):
        b,s, _ = x.size()     #batch_size,sequence_length,input_dim

        #生成Q,K,V, 并分成多个头
        q = self.q(x).view(b,s,self.head,self.dim_per_head_q_k).transpose(1,2)
        k = self.k(x).view(b,s,self.head,self.dim_per_head_q_k).transpose(1,2)
        v = self.v(x).view(b,s,self.head,self.dim_per_head_v).transpose(1,2)

        #注意力分数->(b,head,s,s)
        d = torch.matmul(q,k.transpose(-2,-1))/self.norm
        softmax_out = torch.softmax(d,dim=-1)

```

```
        #加权求和->(b,head,s,dim_per_head_v)
        attention_out = torch.matmul(softmax_out,v)

        #将头的维度合并并返回结果->(b,s,dim_v)
        attention_out = attention_out.transpose(1,2).contiguous().view(b,s,-1)

        #最终输出映射返回 input_dim -> (b,s,input_dim)
        out = self.out(attention_out)
        return out
```

3. 大语言模型中对齐问题的例子

答：对齐问题指模型输出了与人类意图、价值观、事实不一致的内容。

幻觉：生成与事实不符的内容（如编造不存在的学术论文）

指令不遵从：如用户要求中文简短回答，输出英文长篇大论。

过度生成：用户要求一句话总结，输出多段无关内容

偏见歧视/有害内容：性别或种族歧视。暴力、仇恨以及危险指导。

4. 外挂知识库为了解决什么问题

知识时效性不足，垂直领域知识缺失，降低幻觉率。

5. 有哪几种SFT方法

全参数微调：更新模型的全部权重，适合数据量大、算力足的场景，效果好但是成本高。

参数高效微调(PEFT)：仅更新部分参数，当前主流的有：

LoRA/QLoRA:在注意力层插入低秩矩阵，冻结主干参数，训练参数少，使用显存低。

Adapter-tuning:在Transformer层插入小型适配器（adapter）,轻量化适配。

prefix-tuning/P-tuning:在输入前添加可训练前缀，引导模型适应任务，适合多任务场景。

6. LangChain的介绍

答：langchain是专注于大模型开发的应用框架，为了简化大模型与外部数据，接口，工具的集成。核心是通过“组件化”和“链式调用”构建更复杂，更实用的应用。主要模块有以下：

1. 模型(models)：封装各类模型，支持api调用和本地部署模型。
2. 提示词（prompts）：提供提示词模版，提示词管理工具，支持动态参数注入。包含提示词优化工具（如少样本示例，思维链提示），提升模型质量。
3. 链（Chains）:将多个组件按流程串联，实现复杂任务。

4. 数据连接(Data connection)：文档加载器、文本分割器、嵌入、向量存储。
5. 工具(Tools)：让模型调用外部工具（数据库查询，接口，代码实现），实现功能拓展。
6. 代理(Agent）：让模型具有自动选择工具的能力，根据用户需求，自主选择工具，规划步骤，执行任务并返回结果。
7. 记忆(Memory)：存储、提取和更新对话历史，确保模型在多轮交互中能基于上下文生成回应。

7. 模型在训练和推理时占用的显存约多少

1. 训练时：显存主要包含4部分（模型参数，优化器状态，梯度，中间激活值），以FP16为例

	模型参数	优化器状态	梯度	中间激活值
全量	2✖参数量（FP16每个参数占2个字节）	使用Adam,需要4✖参数量（存储一阶矩，二阶矩，fp32格式）。 使用8位优化器，降至2✖参数量。	2✖参数量（FP16）	随批量大小，序列长度增长。
lora	同上（参数冻结）	仅 LoRA 参数（通常占原模型 0.1%-1%）	仅 LoRA 参数的梯度	与全量相同

2. 推理时：显存主要包含2部分
   a. 模型参数：2✖参数量
   b. KV缓存：随序列长度增长（2✖批量✖序列长✖隐藏维度✖数据类型字节数）

对比总结（以 6B 模型为例）

场景	精度	模型参数	优化器状态	梯度	激活值	KV 缓存	总显存
训练	FP16	12GB	24GB	12GB	~~10-20GB~~	-	**58-68GB**
推理	FP16	12GB	-	-	-	2-4GB	**14-16GB**
推理（量化）	INT8	6GB	-	-	-	2-4GB	**8-10GB**

8. 自注意力

自注意力的核心思想

核心：通过Query、Key、Value的矩阵交互，动态计算出序列中每个位置和其它位置的关联权重，捕获长距离依赖关系。

公式：$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$其中，$Q=XW_Q$，$K=XW_K$，$V=XW_V$，$d_k$为 Key 维度。

自注意力的计算步骤是怎样的？

步骤：

① 输入序列X通过线性变换生成Q、K、V；

② 计算Q与K^T的点积，得到相似度矩阵；

③ 对相似度矩阵缩放（除以$\sqrt{d_k}$）并 Softmax 归一化，得到注意力权重；

④ 权重与V加权求和，输出上下文感知向量。

自注意力和循环神经网络（RNN）的区别？

- 优势：
  - **并行计算**：RNN 需顺序处理序列，自注意力可并行计算所有位置的依赖关系，训练速度更快；
  - **长距离依赖**：RNN 受梯度消失限制，自注意力直接建模任意位置关系。
- 劣势：
  - **计算复杂度**：自注意力为$O(n^2d)$，RNN 为$O(nd^2)$，长序列下自注意力内存占用更高。

自注意力与卷积神经网络（CNN）的对比

CNN是通过卷积核捕捉局部特征，自注意力是建模全局依赖。

缩放点积注意力中 $\sqrt{d_k}$ 的作用是什么？

答：除以 $\sqrt{d_k}$ 是一种数值稳定技巧。当向量维度 d_k 较大时，点积结果方差增大，softmax易进入梯度消失区域，缩放后方差稳定为1，提升训练稳定性。

9. 多头注意力

为什么Transformer使用多头注意力

核心：通过多个头并行计算，捕捉不同子空间的语义信息（如句法、语义），增强模型表达能力。

公式：$\text{MultiHead}(Q,K,V)=\text{Concat}(\text{head}_1,...,\text{head}_h)W_O$其中每个头独立计算注意力，最终拼接后通过线性层输出。

Q和K为什么使用不同的权重矩阵

在自注意力机制中，每个输入向量会经过不同的线形变化（权重矩阵）分别生成Q、K、V（查询，键、值）。Q和K的核心作用是计算“注意力分数”（通过点积 QK^T），反应元素之间的关联信息，而V是用于根据权重矩阵聚合信息。

W_Q 专注于学习“如何从输入中提取查询的特征”

W_K 专注于学习“如何从输入中提取匹配查询的特征”

两者分离可以让模型更灵活的捕捉输入之间更复杂的关联。（例如：同一句话中“苹果”在查询“水果”和查询“品牌”时，应该和不同的键向量产生不同的匹配分数。）

如何Q和K使用相同的权重矩阵（如 $W_Q = W_K$ ），会限制模型对“查询”和‘键’角色的差异化学习能力，降低自注意力机制的表达能力。独立的权重矩阵是自注意力机制有效捕捉上下文依赖的重要设计。

10. 位置编码

为什么Transformer需要位置编码

a. 自注意力机制的位置无关性

   i. 自注意力机制的计算逻辑是，通过计算查询（Q）、键（K）的点积计算输入序列中所有元素的注意力权重，再通过值（V）聚合信息。

   ii. 这个过程中，输入序列的元素顺序不会影响最终的注意力分数计算。例如对输入序列['A','B','C']和['B','A','C'],自注意力机制会认为这两者的关联模式是相同的。但是自然语言中，顺序的改变会导致语义完全不同，所以需要外部方式为模型注入位置信息。

b. 位置编码的作用：为序列元素添加位置标识

   答：位置编码的本质：是给输入序列中每个位置的元素分配一个独有的“位置向量”，并将其与输入的词嵌入向量相加，让模型在学习过程中同时感知到元素的内容和其在序列中的位置。

c. 为什么不直接使用位置索引进行位置编码

   答：直观的想法是位置索引作为位置信息（如0，1，2....）,这样的缺陷是

   1. 当序列长度超过了训练时的长度，新的位置索引对于模型是“未知值”，泛化能力差。

   2. 位置索引随着序列长度增大而增大，可能导致模型对“绝对值大小”敏感，而忽略相对位置。（如位置2，3的差异，与位置1002，1003的差异本应该相似，但是数值大小差距悬殊。）

d. 位置编码为什么选择相加而不是拼接

   i. 维度保持：相加操作保持嵌入维度不变，避免拼接导致的维度膨胀和计算量增加

   ii. 数学等价性：拼接后通过线形变化可以等价于相加，相加更高效

Transformer中位置编码的设计：正弦余弦函数

Transformer原论文采用正弦余弦函数构造位置编码，公式如下（pos是位置索引，i是向量维度索引）

$$\mathrm{PE}_{\text{pos},2i} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$\mathrm{PE}_{\text{pos},2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
，这种设计优势在于：

a. 绝对位置唯一：每个位置pos对应唯一的编码向量，模型可以学习到绝对位置。
b. 相对位置可推：对于两个位置 pos 和 $pos+k$,它们的编码向量满足三角函数的平移关系（如 $\sin(a+k)=\sin a\cos k+\sin k\cos a$ ）,模型可以通过学习捕捉相对位置关系。
c. 长度无关：函数可以无限扩展到任意长度序列，不受训练时长度序列限制。

绝对位置编码和相对位置编码的区别

- **绝对位置编码**：
  - 方法：如正弦余弦编码，每个位置对应唯一固定向量（$PE_{pos,2i}=\sin(pos/10000^{2i/d})$）；
  - 局限：难以泛化到训练长度之外的序列，无法显式表达相对位置关系。
- **相对位置编码**：
  - 方法：在注意力计算中引入位置差值作为偏置项（如 RoPE 通过旋转向量捕捉相对位置）；
  - 优势：适用于长序列，能显式建模 token 间的相对距离。

旋转位置编码（RoPE）的原理是什么

旋转位置编码是一种改进的位置编码技术，通过**旋转向量空间**的方式将位置信息直接融入自注意力计算过程中，与传统位置编码（正弦余弦函数）相比，RoPE能更自然的捕获序列中的相对位置关系，在长序列的任务中表现更优。

RoPE在核心思想：让查询向量（Q）和键向量(K)在点积计算之前，根据它们的x相对位置进行旋转。

a. 旋转操作：对于位置 m 的查询向量 q_m 和位置 n 的键向量 k_n ,通过旋转操作使它们在点积时隐式包含相对位置信息 $m-n$ 。
b. 数学实现：使用复数乘法（或等价的矩阵向量）实现向量旋转，对于位置 m 的向量 x ,其旋转后的向量 $x^{'}$ 满足 $x^{'}=x\cdot e^{i\theta m}$ 。其中 θ 是与维度相关的角度参数，$e^{i\theta m}$ 表示复数旋转。

11. 什么是KV缓存

在注意力机制中，每个token的输出都依赖当前输入序列所有token的 key 和 $value$ 向量。若不使用缓存，每次生成新的token时，都需要计算整个序列（包括历史的token）的key和value,造成大量的重复计算。

KV 缓存的工作原理

1. **首次计算（无缓存）**：
   输入初始序列（如 “今天天气”）时，模型正常计算每个 token 的 Key 和 Value 向量，并将这些向量**缓存**起来（存储在内存中）。
2. **后续生成（使用缓存）**：
   生成新 token（如 “很”）后，新输入序列为 “今天天气很”。此时：
   - 仅需计算**新增 token（“很”）** 的 Key 和 Value 向量；
   - 历史 token（“今天天气”）的 Key 和 Value 直接从缓存中读取，无需重新计算；
   - 将新增的 Key 和 Value 追加到缓存中，供下一次生成使用。
3. **注意力计算**：
   每次生成时，Query 向量（基于当前输入序列的最后一个 token 计算）与**缓存的所有 Key** 计算注意力分数，再与**缓存的所有 Value** 加权求和，得到当前 token 的输出。

KV 缓存是大语言模型（如 GPT 系列、LLaMA、ChatGLM 等）实现高效推理的核心技术之一，广泛应用于聊天机器人、文本续写、代码生成等生成式任务中。

12. Encoder-only模型是什么?Decoder-only模型是什么?

1. Encoder-only模式是仅使用transformer的编码器部分，专注于对输入文本的“理解”和“表达学习”。
   a. 核心特点：
      i. 输入输出关系：输入是完整的文本序列，输出是该序列的特征表示（而非生成新文本）。通常用于文本理解类任务。
      ii. 自注意力机制：编码器中的自注意力是“双向的”，每个词会关注输入序列中所有其它词（包括前后文），能捕捉完整的上下文信息。
      iii. 代表模型：BERT
      iv. 典型应用：文本分类（如情感分析，垃圾邮件检测），语义相似度计算
2. Decoder-only模式是仅使用transformer的解码器部分，专注于“生成新的文本序列”，通过autoregressive(自回归)方式逐步生成输出。
   a. 核心特点：
      i. 输入输出关系：输入通常是一个“提示（prompt）”或部分序列，输出是基于输入扩展的完整序列，用于文本生成类任务。
      ii. 自注意力机制：解码器中的自注意力是“单向的”（每个词只能关注之前已生成的词，避免未来信息泄漏），保证生成的逻辑性。
      iii. 代表模型：GPT、LLaMA
      iv. 典型应用：故事生成、对话系统、代码生成

13. 对比SFT、RLHF、DPO的优劣

14. PPO与DPO的区别

1. PPO是openAI在2017年提出的无模型强化学习算法，用于优化策略网络，使其输出更符合奖励函数的期望，在LLM中，主要用于**基于人类反馈的强化学习**（RLHF）。
    a. 3阶段训练流程：
        i. 预训练：使用大规模文本数据训练语言模型
        ii. 奖励模型训练：通过人类标注比较不同输出的优劣，训练奖励模型（RM）评估生成质量。
        iii. ppo优化：使用RM作为奖励函数，通过PPO算法微调模型，使其生成更符合人类偏好的内容。
    b. 优化目标：最大化奖励函数的期望，同时通过策略裁剪约束新策略与旧策略的差异，避免参数更新过大导致性能崩溃。
2. DPO是2023年提出的直接偏好优化算法，作为RLHF的替代方案，无需显式训练奖励模型，直接基于人类偏好数据优化模型。
    a. 简化训练流程
        i. 预训练：与ppo相同
        ii. 直接优化：直接使用人类标注的“更好-更差”样本对，通过对比学习优化模型，无需中间奖励模型。
    b. 优化目标：直接最大化“更好回复”相对于“更差回复”的对数概率差。

15. LayerNorm vs BatchNorm 的适用场景

1. 对比：
    a. layernorm:归一化逻辑是：对单个样本，在 “所有特征位置” 上计算均值和方差，完全不依赖批次内的其他样本。
        i. 为什么适合批次动态变化的场景？
            1. A：归一化的结果仅由当前样本自身决定，不受批次内其他样本影响，因此批次大小变化时，数据分布稳定.
        ii. 为什么适合序列 / 动态场景？
            1. 特征位置意义不固定：文本中 “第 5 个 token” 在不同样本中可能是完全不同的语义（比如一个是名词，一个是动词），跨样本统计没有意义；
    b. batchnorm:归一化逻辑是：对同一批次内的所有样本，在 “相同特征位置” 上计算均值和方差。
        i. 为什么适合 CNN 训练？

- 批次大小稳定：训练时通常固定批次（如 32、64），不会频繁变化，因此批次内的统计量（均值 / 方差）可靠。
- 特征位置意义固定：图像的每个 `(C, H, W)` 位置（如 “红色通道的 (10,20) 像素”）在不同样本中具有相似的语义（都是图像某个区域的颜色信息），跨样本统计是合理的（相当于对同类特征做 “全局校准”）。

维度	BatchNorm	LayerNorm
统计范围	批次内所有样本的 “相同特征位置”	单个样本的 “所有特征位置”
批次大小影响	强依赖：批次变化会导致统计量剧变	无依赖：批次变化不影响统计量
适用场景	批次固定的场景（如 CNN 训练）	批次动态变化的场景（如序列推理）

2. 为什么transformer中使用LayerNorm而不是BatchNorm

   答：选择哪种归一化方法取决于具体任务和模型架构。例如，Transformer 选择 LayerNorm 而非 BatchNorm，是因为处理变长序列时 LayerNorm 更稳定；而 CNN 在图像任务中 BatchNorm 仍是主流选择

2、RAG 、 Agent

1. RAG文档分块的方式有哪些?

1. 基础分块法（基于结构/规则）
   a. 固定长度分块：
      i. 按固定字符数、词数或token数拆分。为避免拆分完整语义，可设置重叠区域，确保上下文连贯性。
      ii. 优点：实现简单。 缺点：可能切断句子、段落等逻辑单元。
   b. 按自然分隔符分块：
      i. 基于文档的自然结构标记拆分，常见的分隔符包括：
         1. 段落分隔符（换行符\n\n）、句子分隔符（句号、问号、感叹号）
         2. 标题标记（如Markdown的#，HTML的<h1>）、列表标记(-,1.)
         3. 文档格式特定分隔符（如PDF的页面边界、Word的章节划分）
      ii. 优点：保留自然语义单元（句子、段落）。 缺点：当单元过长时，检索效果下降。
   c. 按层级结构分块

    i. 先按大粒度（如章节、页面）结构拆分，再在每个大单元内按小粒度（如段落、句子）拆分，形成层级化块结构。
    ii. 例如：书籍->章节->小节->段落->句子
    iii. 优点：支持多粒度检索（根据需求选择大/小单元）；缺点：实现复杂度高。
2. 语义感知分块法（基于内容相关性）

   这类方法通过分析文本语义，确保拆分后的块内部语义连贯，避免切断逻辑关系，通常依赖NLP模型实现。

   a. 语义相似度分块：
      i. 将文本拆分为小单元，计算相邻单元的语义相似度（如嵌入模型BERT,Sentence-bert生成向量，计算余弦相似度），当相似度低于阈值时，进行拆分。
      ii. 优点：块内语义高度连贯。缺点：依赖嵌入模型，计算成本较高。
   b. 基于主题的分块
      i. 识别文本中的主题边界（通过主题模型（LDA）和预训练模型（BART,GPT）），在主题转换处拆分。
      ii. 例如：一篇文章中从“人工智能”转换到“机器学习”时，在切换点拆分块。
      iii. 优点：块与主题高度匹配，适合主题相关的检索需求。缺点：主题识别准确性受文本复杂度影响。
   c. 基于实体/关系的分块
      i. 针对包含大量实体（如人物、地点、事件）的文本，按实体关联关系拆分，确保同一实体的相关信息集中在一个块中。
      ii. 例如：新闻中关于 “某事件” 的描述涉及多个句子，围绕该事件的实体（时间、地点、人物）合并为块。
      iii. 优点：适合实体密集型文本（如新闻、知识库）。缺点：对非实体类文本适用性较低。
3. 混合分块法（结合规则和语义）

2. RAG为什么需要Rerank（二次重排）

因为一次检索和二次重排能实现的目的不同。一次检索用“宽标准”确保高召回，二次重排是“严标准”确保高价值的排前面。

举例：比如我们的项目中二次重排时将简洁风格和图片风格匹配。这是一个非常“挑剔”的筛选逻辑，可能导致内容相似图片的漏召回。这也是一个计算度复杂的，一次检索大量数据，会拖慢检索速度。

3. 如何设计一个意图识别

定义：意图识别的核心任务就是精准判断用户的需求，并映射到正确的工具调用。

实现主要模块：

1. 建立 意图—工具 映射表（意图—调用工具—所需参数）
2. 进行参数提取：若用户输入没有明确参数，需要通过从实体识别（NER）从上下文提取或主动追问。
3. 意图识别（常见问题）：
   a. **输入模糊或信息不全：意图边界模糊或缺乏所需参数**
      i. 举例：用户说 “帮我订个票” （未说明 “机票 / 火车票” “出发地 / 目的地” ）；或 “最近有什么推荐？” （未说推荐 “电影 / 商品 / 旅游地” ）。
      ii. 解决思路：
         1. 结合对话历史推断：如用户前序说 “我要去上海出差” ，则 “订个票” 可推断为 “上海方向的机票”
         2. 主动追问：用规则或模型判断缺失参数（如 “订机票” 缺少 “日期” 时，自动回复 “请问需要订哪一天的机票？”
   b. **意图重叠或多意图：单标签分类模型无法识别多意图，导致工具调用遗漏。**
      i. 举例：用户说 “帮我查一下明天去上海的机票，顺便看看上海的天气” （同时包含 “查询机票” 和 “查询天气” 两个意图）
      ii. 解决思路：
         1. 多标签分类模型：如同时预识别到“查询机票” 和 “查询天气”两个需求
         2. 意图优先级排序：根据场景定义优先级（如电商场景中 “支付” 优先于 “咨询” ），先调用高优先级工具。
   c. **口语化与噪音干扰：非常规表达导致模型特征提取失效**
      i. 举例：用户输入 “哎，那个啥，昨天买的鞋有点小，能换不？” （包含口语词 “那个啥” 、省略句 “能换不” ）；或有错别字 “帮我查一下明天的天汽” （“汽” 应为 “气” ）
      ii. 解决思路：
         1. 预处理增强：用拼写纠错工具（如 TextBlob）修正错别字，用停用词表过滤 “那个啥” “哎” 等无意义词。
         2. 语义模型优化：采用BERT预训练模型，提升对非标准化表达的理解能力
   d. **长尾意图（低频意图）：训练数据少，模型对长尾意图的识别准确度低（可能 <50%）**
      i. 举例：电商场景中，“查询发票抬头” “修改会员生日” 等意图出现频率低（＜5%），但必须支持。
      ii. 解决思路：

1. Few-shot/Zero-shot 学习：用少量样本（如 10 个）训练模型识别长尾意图（如用 Prompt Tuning 微调 BERT）；
2. 规则补充：对极低频意图，直接用关键词规则匹配（如 “发票抬头” 相关输入，直接调用发票接口）。

e. **领域迁移问题：模型的泛化能力不足，对新领域的语义理解弱**

i. 举例：原模型在 “电商场景” 训练，迁移到 “金融场景” 后，对 “查询信用卡账单” 等新意图识别效果差。

ii. 解决思路：

1. 领域自适应：用新领域的少量数据微调预训练模型（如用金融数据微调 BERT）；
2. 通用模型+领域词典：在通用模型基础上，加入领域专属词典（如金融领域的 “信用卡” “账单”），增强特征权重。

4. 嵌入模型常见的三种检索功能：密集检索、多向量检索和稀疏检索

- 密集检索：用深度学习模型将文本转为高维向量，靠向量相似度匹配，能理解语义（如识别同义词），但耗资源、成本高。
- 多向量检索：密集检索的扩展，细粒度匹配文本片段，长文档检索更精准，但计算复杂、耗时。
- 稀疏检索：基于关键词匹配，用词频、逆文档频率等统计方法，简单快、成本低、可解释，但不懂语义，依赖词汇直接匹配。

5. 如果rag效果不好，怎么精准定位问题

当 RAG（检索增强生成）效果不佳时（如回答不准确、遗漏关键信息、出现幻觉等），需要从检索（Retrieval）、生成（Generation）、数据链路（Data Pipeline） 三个核心环节拆解

1. 检索阶段：

a. 召回不足的可能原因：

i. 分块不合理、嵌入模型不匹配、召回策略单一

b. 排序不准确的可能原因：（mrr低）

i. 相似度计算逻辑不合理、多路召回融合策略不当、噪声文档干扰

c. 分块是否合理

i. 是否存在关键信息被拆分到多个块中

ii. 是否分块过大

iii. 分块的语义连贯性

2. 生成阶段：

    a. 上下文是否被模型理解：回答是否包含检索上下文
    b. 是否存在幻觉：回答有上下文不存在的信息
        i. 原因：上下文信息不足，提示词设计缺陷，模型本身幻觉倾向
    c. 逻辑推理是否正确

6. 归一化在RAG中的作用
7. 为什么选择向量数据库，而不是传统数据库
8. 什么是A2A（Agent2Agent）

3、大模型、prompt

1. 什么是过拟合，欠拟合

维度	过拟合	欠拟合
训练集表现	极好（损失低、准确率高）	差（损失高、准确率低）
测试集表现	差（泛化能力弱）	差（未学到核心规律）
核心原因	模型“学太多”（含噪声）	模型“学不够”（能力不足）
解决核心思路	限制学习（正则化、早停等）	增强学习（扩容、调优训练）

2. 什么是大模型的 epoch 和 学习率
3. 微调时模型遗忘怎么解决
4. 如何保证大模型生成内容的合规性
5. vllm的推理加速是怎么实现的
6. 模型蒸馏和模型量化

4、部署

ollama:

如何将Python中实现的文生图流程，打包成接口，如何部署发布，然后让java项目可以调用